

CS1CA Computer Systems Architecture

Memory hierarchy Design (3)

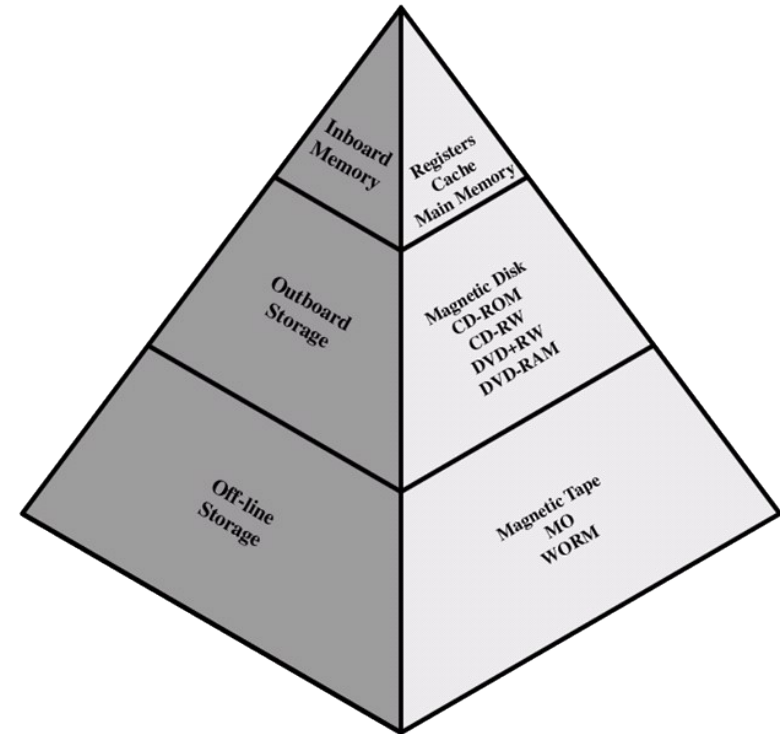
Week 11 lecture 1 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher



Continue our discussions on the topic
of memory hierarchy design.

In this lecture

- The following contents will be delivered
 - Techniques for reducing cache misses
 - Techniques for reducing hit time
- The learning outcomes are listed below
 - To gain knowledge of possible cache misses
 - To be able to use introduced techniques to reduce cache misses and cache hit time for improving cache performance

Three types of cache misses

- There are three types of cache misses
 - **Compulsory**: the very first access to a block, which cannot be in the cache, brings the block into the cache from RAM;
 - **Capacity**: the cache cannot contain all the blocks needed during the execution of a program. This causes blocks to be discarded and later retrieved;
 - **Conflict**: this applies to set associative and direct mapped caches. Conflict misses cause a block to be discarded and later retrieved if too many blocks map to its set.

Example: For an 8-block cache, block addresses 12, 20, 28, ... $\text{MOD } 8 = 4$. If these addresses are accessed, block 4 in cache will be used frequently. Conflict misses occur.

Technique 1 to reduce miss rate: larger block size

- ✓ Reduce compulsory misses – taking advantage of spatial locality;
- ✧ Increase the miss penalty as more data to be transferred for each memory reference;
- ✧ Increase conflict and capacity misses, thus increase miss rate;
- To minimise both miss rate and miss penalty, the influence of RAM latency and bandwidth is taken into account.

Technique 2 to reduce miss rate: larger caches

- This technique is to increase capacity of the cache, and to reduce capacity misses.
- The drawback is longer hit time and higher cost.
- This technique is popular in off-chip caches, e.g. second-level caches.

Technique 3 to reduce miss rate: higher associativity

✓ Two rules of thumb

- Eight-way set associative is as effective in reducing misses for these sized caches as fully associative.
- The 2:1 cache rule of thumb states that a direct-mapped cache of size N has about the same miss rate as a 2-way set-associative cache of size $N/2$.

✦ However, increasing associativity can come at the price of increasing hit time. The pressure of fast processor clock cycle encourages the simple cache design.

Technique 4 to reduce miss rate: pseudo-associative caches

- Objective: to get the miss rate of set-associative caches and the hit speed of direct mapped caches.
- Cache access is same as in the direct-mapped cache for a hit;
- On a miss, the cache is probed again, e.g. invert the most significant bit of the index to find the block in the pseudo-set;
- ✗ Total miss penalty may be increased.
- Attractive on paper but variable hit time complicates a pipelined CPU design.

Technique 5 to reduce miss rate: compiler optimisations -1

- This method is **a software only method** without any hardware changes;
- Code can be rearranged without affecting program correctness, e.g. reordering the procedures of a program might reduce instruction miss rates by reducing conflict misses;
- Rearrange data to improve the spatial and temporal locality;
- Merging arrays into a data structure ensures better spatial locality and is good software engineering practice.

Technique 5 to reducing miss rate: compiler optimisation -2

- **Loop Interchange**

- Loop is interchanged to make the code access the data stored in memory in the order. This reduces misses by improving spatial locality. Reordering maximises use of data in a cache block before it is discarded.

original code

```
for(j=0; j<100; j++)  
    for(i=0; i<5000; i++)  
        x[i][j]=2*x[i][j]
```

new code

```
for(i=0; i<5000; i++)  
    for(j=0; j<100; j++)  
        x[i][j]=2*x[i][j]
```

- The first version skips through memory in strides of 100 words. Revised version accesses all the words in the cache block before going to the next one. Cache performance is improved without affecting the number of instructions.

Technique 5 to reduce miss rate: compiler optimisation -3

- **Blocking**

- The blocking algorithm operates on sub matrices of blocks instead of operating on entire rows and columns of an array. The goal is to maximise accesses to the data loaded into the cache before the data are replaced. This method is based on improved temporal locality.

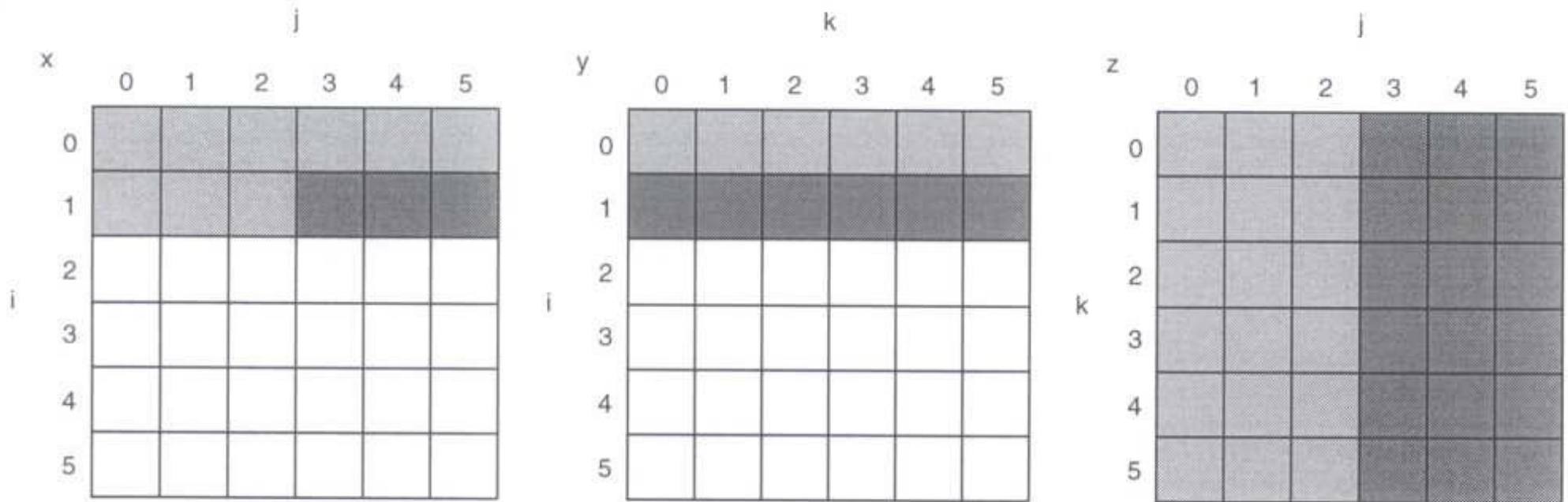
Original C code: 3 arrays x, y, z

```
For(i=0; i<N; i=++) {  
    for(j=0; j<N; j=++) {  
        r=0;  
        for(k=0; k<N; k=++) {  
            r=r+y[i][k]*z[k][j];  
        }  
        x[i][j]=r;  
    }  
}
```

(Blocking continue...)

- The age of array accesses is indicated by shade; white means not yet touched, light means older accesses and dark means newer accesses.

A snapshot of 3 arrays x , y , and z when $i=1$



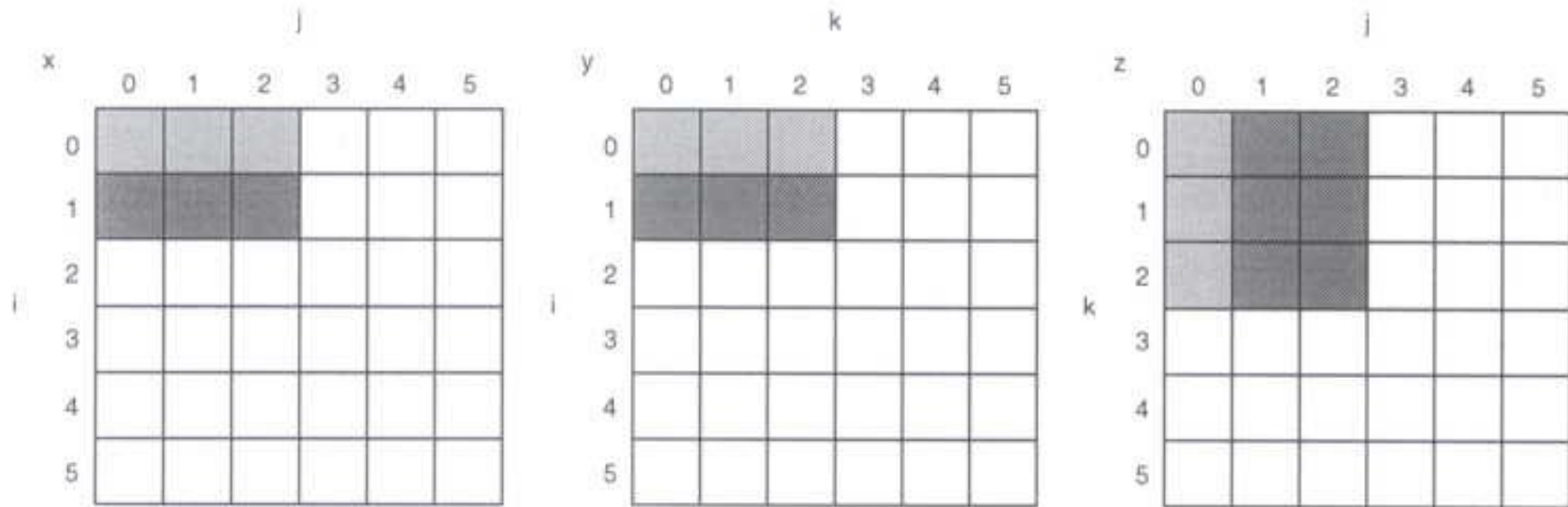
Blocking continue...

- The 2 inner loops read all $N \times N$ elements of array z , access the same N elements in a row of array y repeatedly and writes to one row on N elements of array x . No. of capacity misses depends on N and the size of the cache. If the cache can hold all 3 ($N \times N$) matrices, there is no problem (*refer to the figure in the last slide*);
- Otherwise the code is changed to compute on a submatrix of size $B \times B$. The inner loops compute in steps of B rather than going from the beginning to the end of the arrays. (Pseudo code)

```
for(jj=0; jj<N; jj=jj+B)
  for(kk=0; kk<N; kk=kk+B)
    for(i=0; i<N; i++)
      for(j=jj; j<min(jj+B-1,N); j++) {
        r=0;
        for(k=kk; k<min(kk+B-1, N); k++) {
          r = r+y[i][k]*z[k][j];}
        x[i][j]=x[i][j]+r;
      };
```

Blocking continue...

Figure illustrating the accesses to the three arrays using blocking with block factor $B=3$.



- **Note:** the smaller number of elements has been accessed.
- It uses the combination of spatial (array y) and temporal (array z) locality.

Reducing hit time - 1

- Hit time is critical because it affects the CPU clock rate;
- Technique one: using small and simple caches
 - Smaller hardware is faster;
 - To keep the cache small enough to fit on the same chip, avoid time penalty of going off-cache;
 - Keep the cache simple, such as direct mapped cache.

Reducing hit time - 2

- Technique two: avoiding address translation during indexing of the cache.
 - Cache must cope with the translation of a virtual address from the CPU to a physical address to access memory;
 - Virtual caches with virtual addresses are used to eliminate address translation time from a cache hit.

Reducing hit time – 3:

- Technique three: **pipelining writes**
 - Write hits take longer time than read hits;
 - **Pipelining writes**
 - Tags and data are split so that they can be addressed independently;
 - The cache compares the tag with the current write address, and writes to the data portion of the previous write which has been determined to be a hit;
 - Therefore the write hits are improved.

Cache optimisation summary

- Please refer to Figure 5.26 in the Hennessy and Patterson's book, 3rd edition (page 449) to review techniques for cache optimisations and their impact on cache performance.
- For the 5th edition of the same book, it is shown in Figure 2.11 on page 96.

End of this lecture

Questions and exercises

- Articulate the 5 techniques for reducing miss rate and the 3 techniques for reducing hit time.